Plagiarism Detection using String Matching Algorithms

Design and Analysis of Algorithms | 5th Semester | Mini Project

Group number: 13

Students and enrollment numbers: Patel Moxa – 12402080601085

Patel Rishi – 12402080601118

Patel Vrusha - 12402080601155

Faculty: Yamini Dubey

Problem statement

	Description
Problem	Find the passages of a submitted text that are copied word for word from a source text.
Input	Submitted text S, source text T, and k, the shortest phrase length in words (3 to 10).
Output	Overlap percentage of each text, both texts with shared phrases marked, a list of passages, and cost per algorithm.
Constraints	Exact matches only, after lowercasing and removing punctuation. Reworded text is not detected.
Objective	Detect every shared phrase and compare Naive, Rabin–Karp and KMP on comparisons and time.

Motivation and real-world application

Why it matters

- Copied work is hard to spot by eye
- Long documents make manual checks slow
- Credit and honest work depend on it

Where it is used

- Universities checking assignments
- Publishers checking submissions
- Code and document version audits

Who benefits

- Teachers and examiners
- Editors and reviewers
- Students checking their own citations

Objectives

- Define the plagiarism problem as exact phrase matching of k or more words
- Implement three string matching algorithms: Naive, Rabin–Karp and Knuth–Morris–Pratt
- Report the overlap percentage and mark every shared phrase in both texts
- Measure comparisons and time on small, medium and large inputs
- Compare the measured results with best, average and worst-case theory

Existing approach and background

Existing approach

- Read both texts side by side
- Search one phrase at a time with a basic scan
- Fingerprint tools hash many phrases

Limitations

- Manual reading misses phrases and takes hours
- A basic scan can take O(n·m) time
- Semantic tools need far more than string matching

Need

- A fast, exact and repeatable search
- A comparison of algorithms to show which scales best

Proposed approach



Input texts	Normalize	k-word phrases	Search	Mark words	Report
Submission and source plus k	Lowercase and strip punctuation	Every run of k consecutive words	Naive, Rabin–Karp or KMP at word boundaries	Every word covered by a match	Percentages, passages and cost

The same process runs again with the texts swapped, so the page reports how much of each text appears in the other.

Algorithms used

Naive

- Idea: Slides the pattern one step at a time and compares left to right.
- Why: The baseline. Simple and needs no extra memory.
- Data: Arrays of words

Rabin–Karp

- Idea: Compares a rolling hash of each window, and checks characters only when hashes agree.
- Why: Hashing scales to many phrases, as fingerprint tools do.
- Data: Hash values

Knuth–Morris–Pratt

- Idea: Uses a prefix table so the scan never moves backwards in the text.
- Why: Guarantees linear time on every input.
- Data: Prefix table

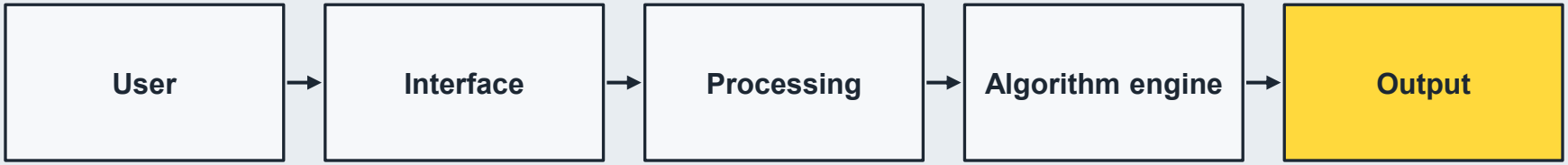
Algorithm pseudocode

```
KMP search
lps = prefix table of P
j = 0
for i = 0 to n-1
  while j>0 and T[i]!=P[j]
    j = lps[j-1]
  if T[i]=P[j]: j = j+1
  if j=m: report i-m+1
     j = lps[j-1]
```

```
Rabin–Karp search
hp = hash(P)
ht = hash(T[0..m-1])
for i = 0 to n-m
  if hp=ht and T[i..i+m-1]=P
    report i
  ht = roll(ht, T[i], T[i+m])
```

KMP: O(n + m) time, O(m) space. Rabin–Karp: O(n + m) average, O(n·m) worst, O(1) space. Naive: O(n·m) worst, O(1) space.

System design



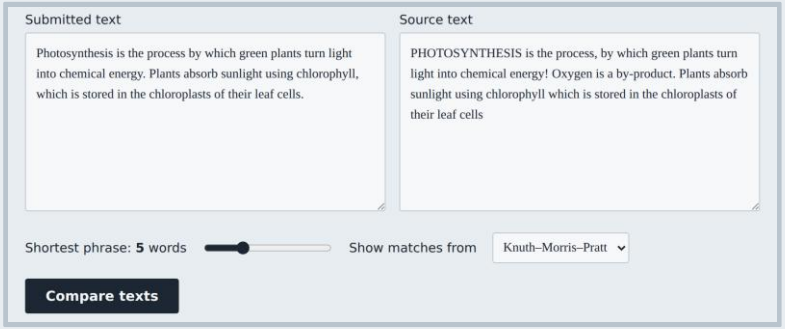
User	Interface	Processing	Algorithm engine	Output
Pastes texts and picks k and the algorithm	Browser page with text boxes and tabs	Tokenizer and k-word phrase builder	Naive, Rabin–Karp and KMP functions	Marked texts, percentages, passages, benchmark table

Everything runs in the browser. No text is sent to a server.

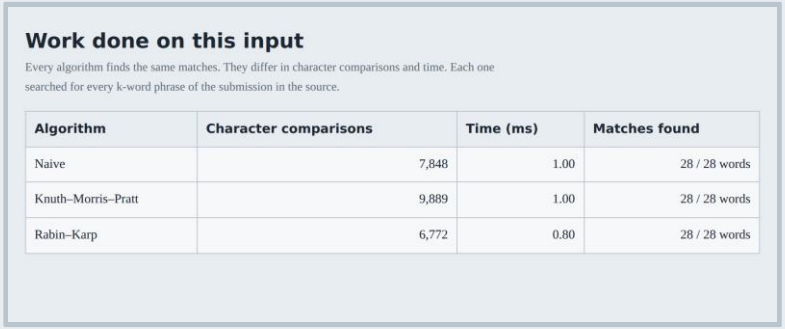
Implementation

Item	Details
Language	JavaScript, HTML and CSS
Libraries	None. Fonts load from Google Fonts.
Modules	Tokenizer, three search functions, phrase checker, marking and scoring, benchmark runner
Data structures	Arrays of word positions, prefix table for KMP, rolling hash numbers, boolean array of covered words
Run	Open index.html in Chrome. No install or server needed.

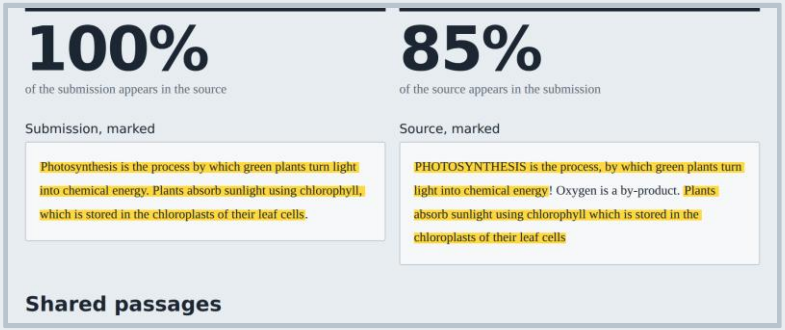
Project screenshots



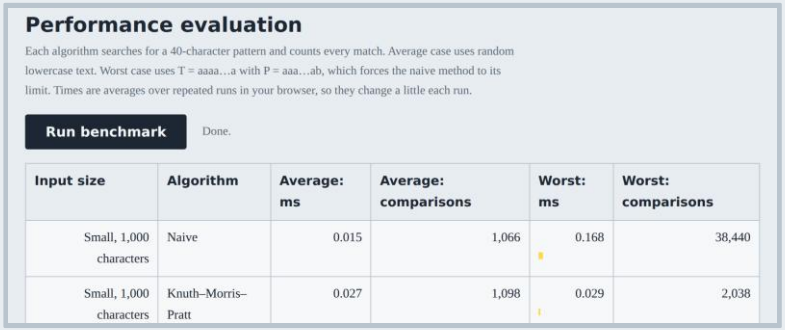
Input screen: two texts and phrase length entered



Processing screen: comparisons and time per algorithm



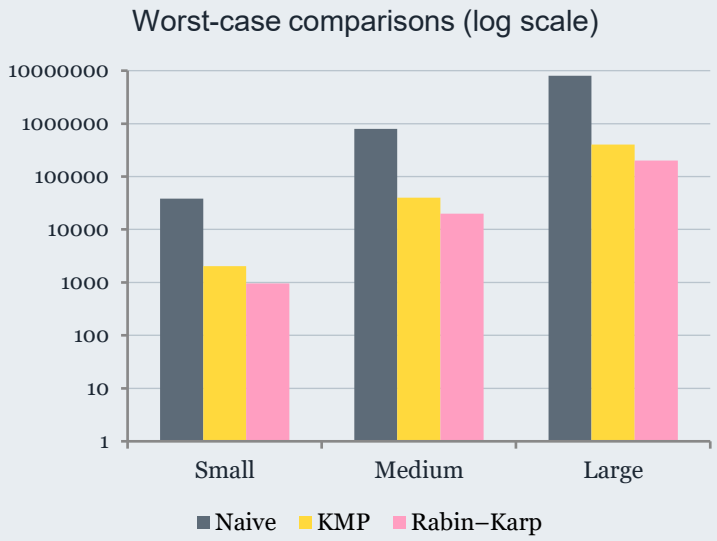
Output screen: shared phrases marked in both texts



Performance screen: benchmark for three input sizes

Results

Input size	Algorithm	Comparisons	Time (ms)
Small (1,000)	Naive	38,440	0.31
Small (1,000)	KMP	2,038	0.02
Small (1,000)	Rabin–Karp	961	0.02
Medium (20,000)	Naive	798,440	7.22
Medium (20,000)	KMP	40,038	0.48
Medium (20,000)	Rabin–Karp	19,961	0.19
Large (200,000)	Naive	7,998,440	73.16
Large (200,000)	KMP	400,038	4.57
Large (200,000)	Rabin–Karp	199,961	1.92



Worst case, pattern length 40. Times were measured in Node.js with the same code as the website, and will differ a little on your machine. Run the website benchmark and use your own numbers for the final report.

Complexity analysis

Algorithm	Best case	Average case	Worst case	Space
Naive	O(n)	O(n)	O(n·m)	O(1)
Rabin–Karp	O(n + m)	O(n + m)	O(n·m)	O(1)
KMP	O(n + m)	O(n + m)	O(n + m)	O(m)

- Theory against practice at 200,000 characters and m = 40:
- Naive did 7,998,440 comparisons. Theory says n·m = 8,000,000.
- KMP did 400,038, close to 2n = 400,000.
- Rabin–Karp did 199,961, near n, because the pattern hash never matches. Its O(n·m) case needs many hash collisions.
- On random text all three stay near n comparisons.

Advantages, limitations and future scope

Advantages

- Finds every exact shared phrase
- Compares three algorithms side by side
- Runs in the browser with no upload
- No installation or libraries

Limitations

- Reworded text is missed
- Very large texts are slow in a browser
- Exact matches only, no meaning check

Future scope

- Search many sources at once
- Fingerprinting by winnowing
- File upload for documents
- Suffix arrays for faster search

Conclusion and references

- Problem: find copied phrases between two texts.
- Solution: Naive, Rabin–Karp and KMP applied to every k-word phrase.
- Finding: KMP stays linear on every input, while Naive and Rabin–Karp can reach O(n·m).

References

Cormen, Leiserson, Rivest, Stein. Introduction to Algorithms. String matching chapter.
Knuth, Morris, Pratt (1977). Fast pattern matching in strings.
Karp and Rabin (1987). Efficient randomized pattern-matching algorithms.
Sedgewick and Wayne. Algorithms. Substring search.